

Heterogeneous Graph Transformer (HGT)

Hu et al., 2020

Sysname DSL

Paper Equations

Initial embedding

$$H_{\text{Papers}}^{(0)}(v) = \text{ReLU}(\text{Linear}_{\text{Papers}}(x_v))$$

```
// One rule per node type (DBLP has 4); Papers shown:
HGT<'Papers',0>(v; ReLU(Linear(F, d)(x_v))) :- Papers(v; x_v) .
```

Learnable parameters

Templated by layer L , node type τ ,
edge type ϕ , and head i :

$$\begin{aligned} \text{a } & \mathbf{KLin}_{\tau}^i, \mathbf{QLin}_{\tau}^i, \mathbf{MLin}_{\tau}^i \in \mathbb{R}^{d \times d/h} \\ \text{b } & \mathbf{W}_{\phi}^{\text{ATT}}, \mathbf{W}_{\phi}^{\text{MSG}} \in \mathbb{R}^{d/h \times d/h} \\ \text{c } & \mu_{\langle \tau_s, \phi, \tau_t \rangle}, \text{skip}_{\tau(t)}^{(L)} \in \mathbb{R} \\ \text{d } & \mathbf{ALin}_{\tau} \in \mathbb{R}^{d \times d} \end{aligned}$$

```
a KLin<L,tau_s,i> = Linear(d, d/h) .
  QLin<L,tau_t,i> = Linear(d, d/h) .
  MLin<L,tau_s,i> = Linear(d, d/h) .
b W_ATT<L,phi,i> = Parameter(d/h, d/h) .
  W_MSG<L,phi,i> = Parameter(d/h, d/h) .
c Mu<L,phi,i> = Parameter(1) .
  Skip<L,tau_t> = Parameter(1) .
d ALin<L,tau_t> = Linear(d, d) .
```

Attention head

$$\begin{aligned} \text{1 } & K^i(s) = \mathbf{KLin}_{\tau(s)}^i(H^{(L-1)}[s]) \\ \text{2 } & Q^i(t) = \mathbf{QLin}_{\tau(t)}^i(H^{(L-1)}[t]) \\ \text{3 } & \text{ATT}^i = \text{softmax}_i \left(\frac{K^i W_{\phi(e)}^{\text{ATT}} Q^{iT} \cdot \mu}{\sqrt{d/h}} \right) \end{aligned}$$

```
def ATT_Head<L,tau_s,phi,tau_t,i>():
  1 K(s; KLin<L,tau_s,i>(z)) :- HGT<tau_s,L-1>(s; z) .

  2 Q(t; QLin<L,tau_t,i>(z)) :- HGT<tau_t,L-1>(t; z) .

  3 Dot(s,t; (z_k @ W_ATT<L,phi,i> @ z_q.T) * Mu<L,phi,i> / sqrt(d/h)) :-
    K(s; z_k), phi(s,t), Q(t; z_q) .
  ATT(s,t; z) :- Softmax(Dot)(s,t; z) .
enddef
```

Message head

$$\text{4 } \text{MSG}^i(s, e, t) = \mathbf{MLin}_{\tau(s)}^i(H^{(L-1)}[s]) W_{\phi(e)}^{\text{MSG}}$$

```
def MSG_Head<L,tau_s,phi,tau_t,i>():
  4 M(s; MLin<L,tau_s,i>(z)) :- HGT<tau_s,L-1>(s; z) .
  MSG(s,t; z_m @ W_MSG<L,phi,i>) :- M(s; z_m), phi(s,t) .
enddef
```

Target-Specific Aggregation

$$\begin{aligned} \text{5 } & \tilde{m} = \prod_{i=1}^h (\text{ATT}^i \cdot \text{MSG}^i) \\ \text{6 } & \tilde{H}^{(L)}[t] = \sum_{s \in N(t)} \tilde{m}(s, e, t) \\ \text{7 } & \sigma_g = \text{Sigmoid}(\text{skip}_{\tau(t)}^{(L)}) \\ \text{8 } & H^{(L)}[t] = \sigma_g \cdot \mathbf{ALin}_{\tau(t)}(\text{GELU}(\tilde{H}^{(L)})) \\ & + (1 - \sigma_g) \cdot H^{(L-1)}[t] \end{aligned}$$

```
def WeightedMessageAgg<L,tau_s,phi,tau_t>():
  5 WMsg<i>(s,t; z_att * z_msg) :-
    ATT_Head<L,tau_s,phi,tau_t,i>(s,t; z_att),
    MSG_Head<L,tau_s,phi,tau_t,i>(s,t; z_msg) .
  m_tilda(s,t; Concat(*z)) :- WMsg<i>(s,t; z), ... [i = 1 to h] .
enddef

def HGT<tau_t,L>():
  6 H_tilda(t; sum(z)) :-
    WeightedMessageAgg<L,tau_s,phi,tau_t>(s,t; z) |... [N(tau_s,phi,tau_t)] .
  7 sg<L,tau_t> = Sigmoid(Skip<L,tau_t>) .
  8 H(t; sg<L,tau_t> * ALin<L,tau_t>(GELU(z)) + (1 - sg<L,tau_t>) * z2) :-
    H_tilda(t; z), HGT<tau_t,L-1>(t; z2) .
enddef
```

Figure: HGT (Hu et al., 2020) — paper equations (left) and corresponding Sysname DSL (right). Each row shows one component of the architecture, demonstrating the one-to-one correspondence between the mathematical formulation and executable DSL code. In the DSL, ϕ denotes the edge type ($\phi(e)$ in the paper). Supplementary material for “Incorporating Deep Learning Design in Database Queries.”